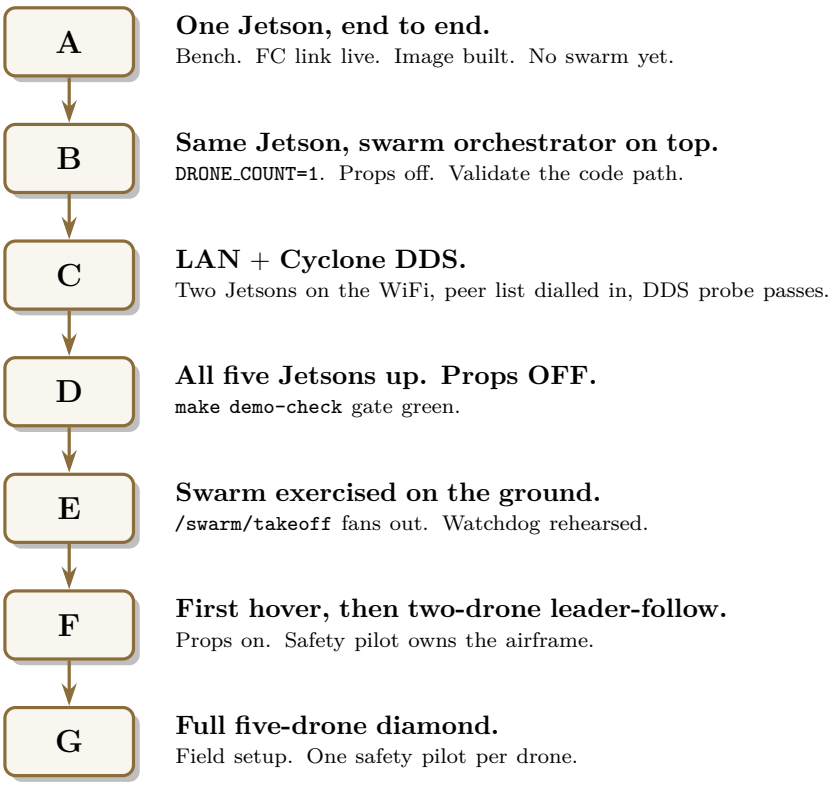
Orynth v2

Bring-Up Sequence: Bench to Swarm



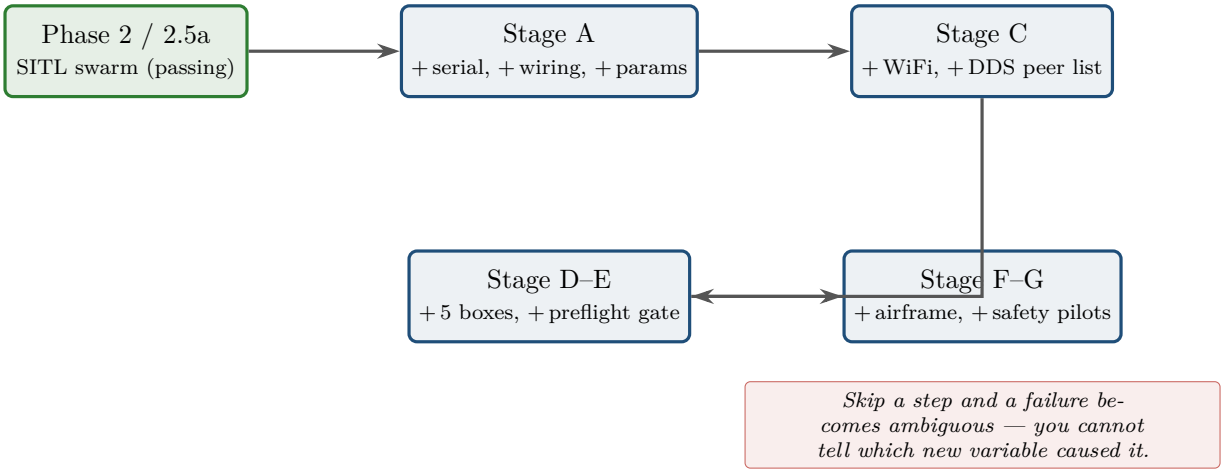
Companion to:
`docs/architecture/end_to_end/end_to_end_setup.pdf` – architecture.
`docs/runbooks/jetson_swarm_operations.md` – per-command reference.

*This document is the **operational, risk-ordered** view: what to do, in what order, with what gate at each step.*

Contents

1. The principle: earn each new variable

The SITL stack proves software, not hardware. Moving to airframes adds five new failure modes the simulator cannot reproduce: serial wiring, FC parameter drift, the wireless LAN, multi-drone DDS discovery, and the airframe itself. Each stage in this document is sized to add *one* of those at a time, with a gate at the end before the next stage starts.



The two reference documents. Throughout this guide:

- `docs/architecture/end_to_end/end_to_end_setup.pdf` — the big picture: what runs where, why.
- `docs/runbooks/jetson_swarm_operations.md` — the per-command reference with the troubleshooting table.

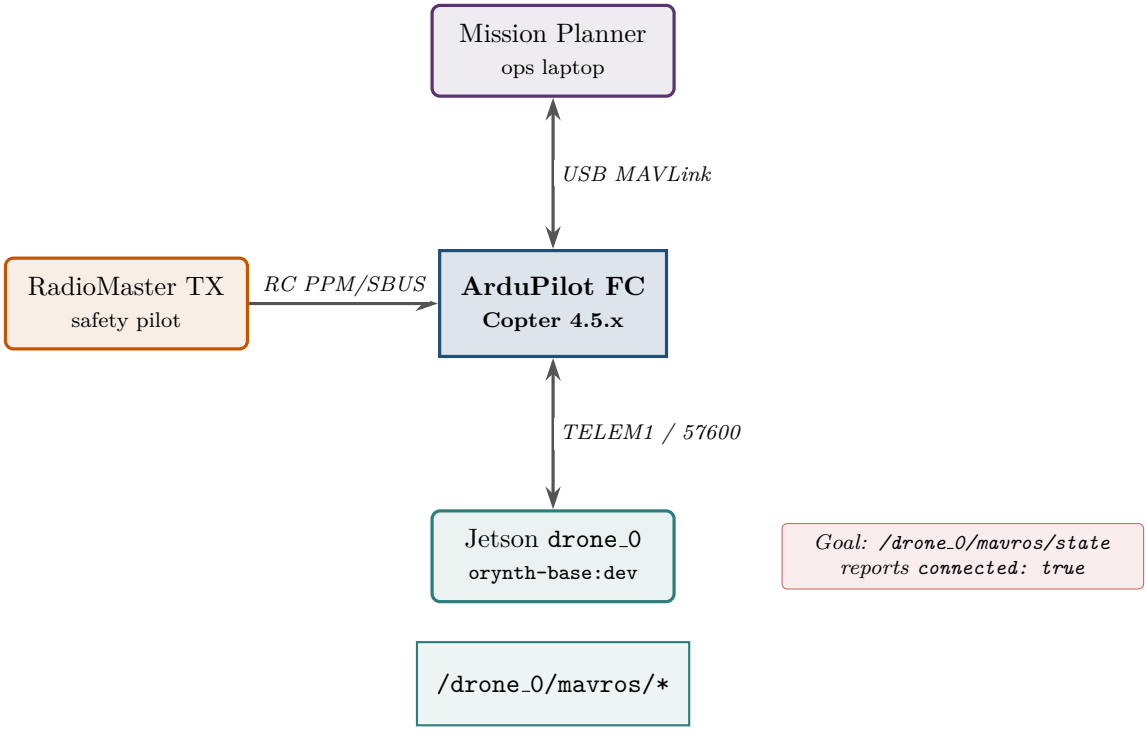
This document sequences *when* to do each thing, and what ”pass” looks like before you move on.

2. Stage A — One Jetson, end to end

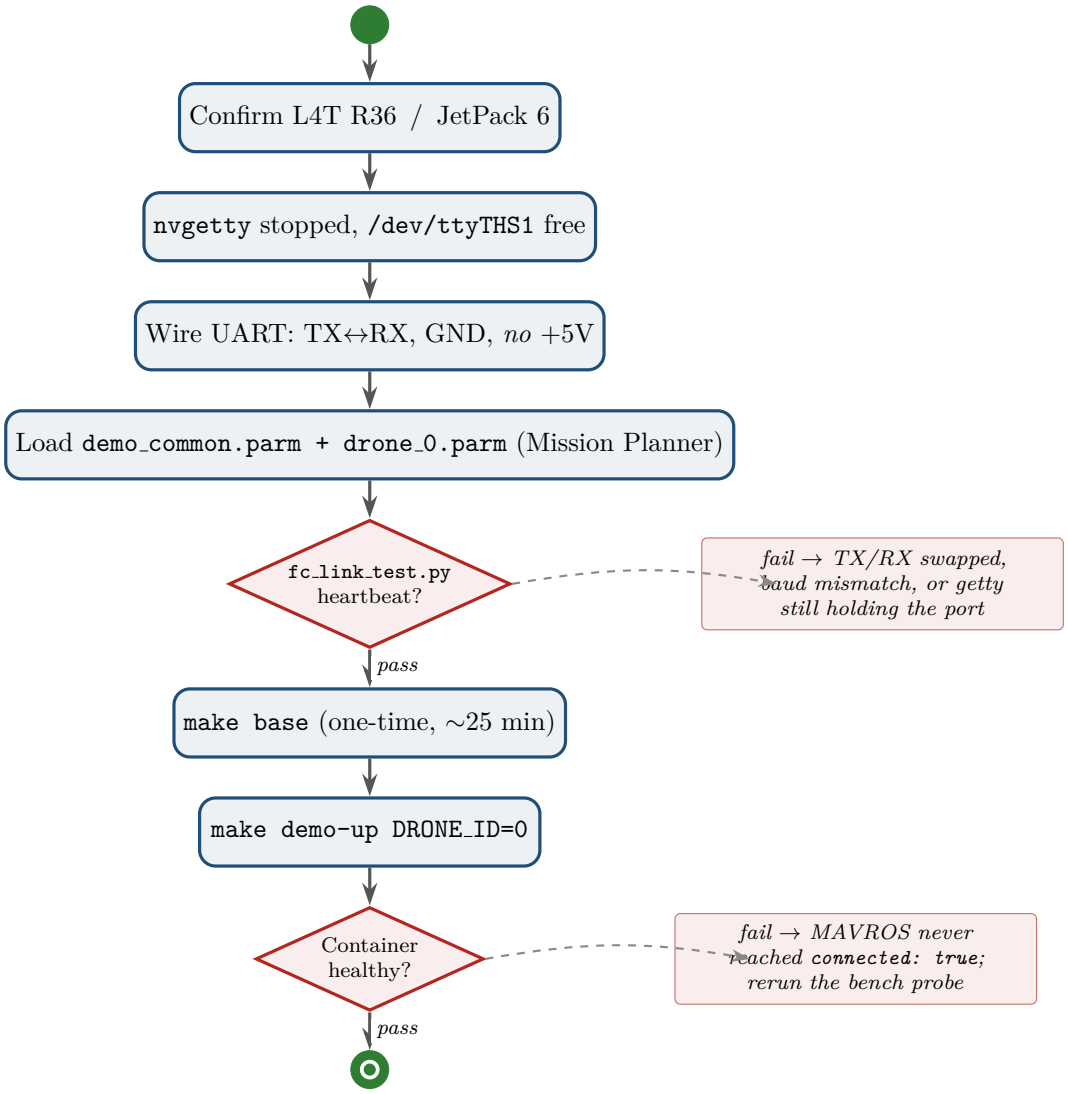
Stage A — Single Jetson, single FC, on the bench

Goal: a single companion talks to a single ArduPilot FC over MAVROS, with the base image built and healthcheck green.

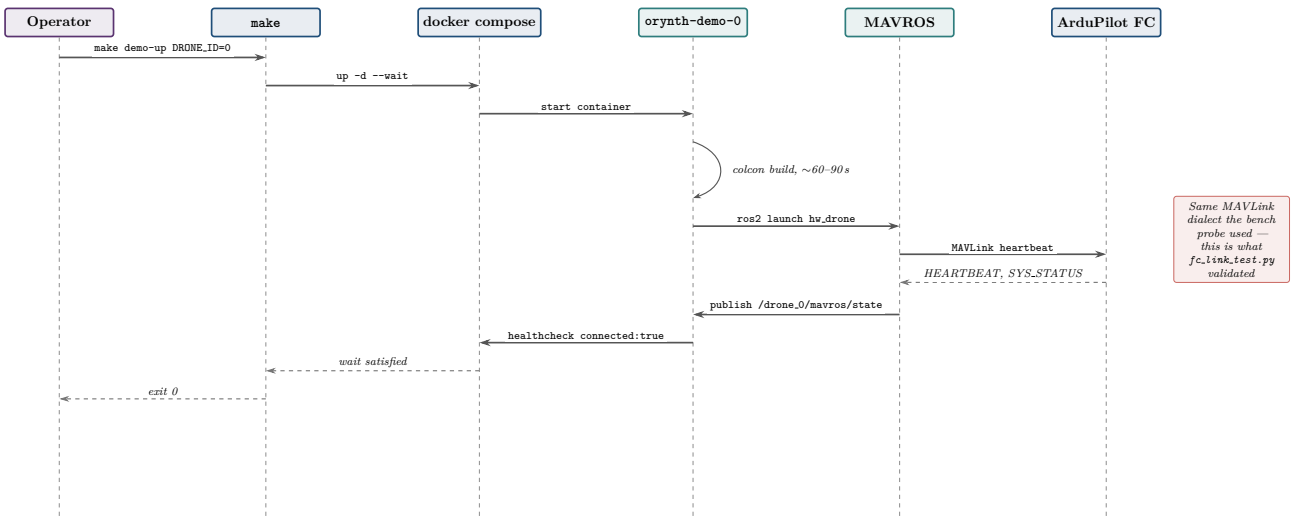
2.1. Deployment view



2.2. Bring-up flow



2.3. What happens inside, on `make demo-up`



Gate to leave Stage A: `ros2 topic echo --once /drone_0/mavros/state` inside `orynth-demo-0` prints `connected: true`.

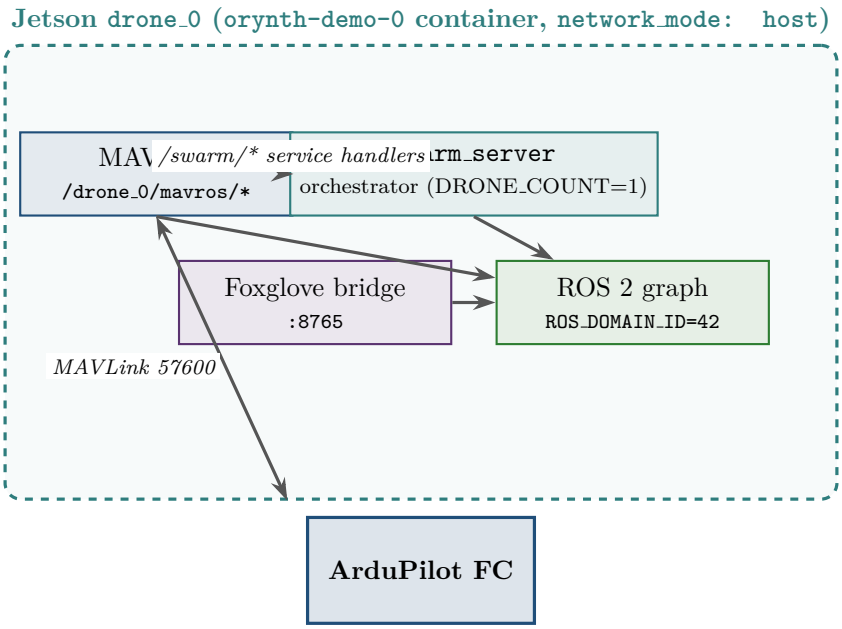
3. Stage B — Add the swarm orchestrator

Stage B — Same Jetson, `swarm_server` on top, props off

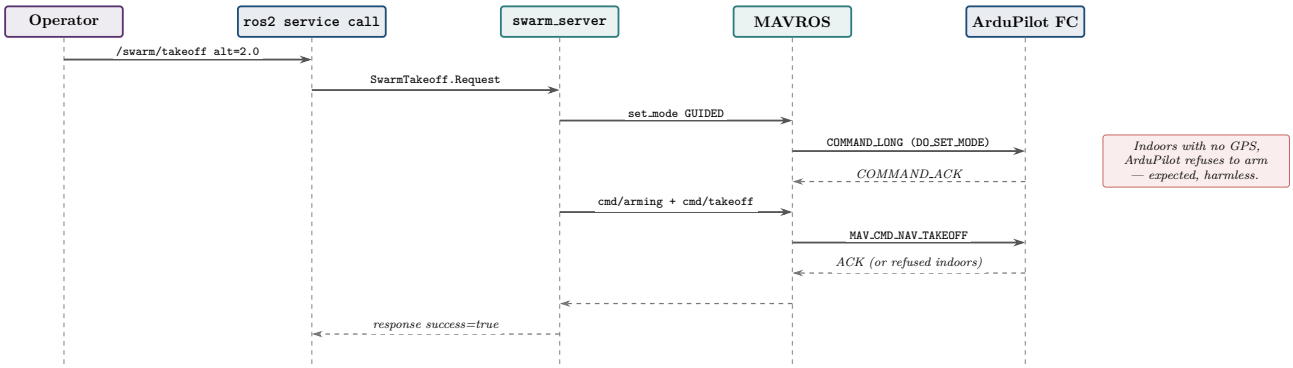
Goal: the orchestrator code path runs end-to-end on real hardware before the network is involved.

The single-drone case is also a *degenerate swarm*: launch with `DRONE_COUNT=1` and `swarm_server` orchestrates exactly one airframe. This validates the whole code path without involving WiFi.

3.1. Component view — what runs on one Jetson



3.2. One service call, end to end



Gate to leave Stage B: the FC switches to GUIDED, `/swarm/status` walks through the takeoff phases, and `/swarm/land` returns it to STABILIZE. Props off; arming indoors will fail and that is fine.

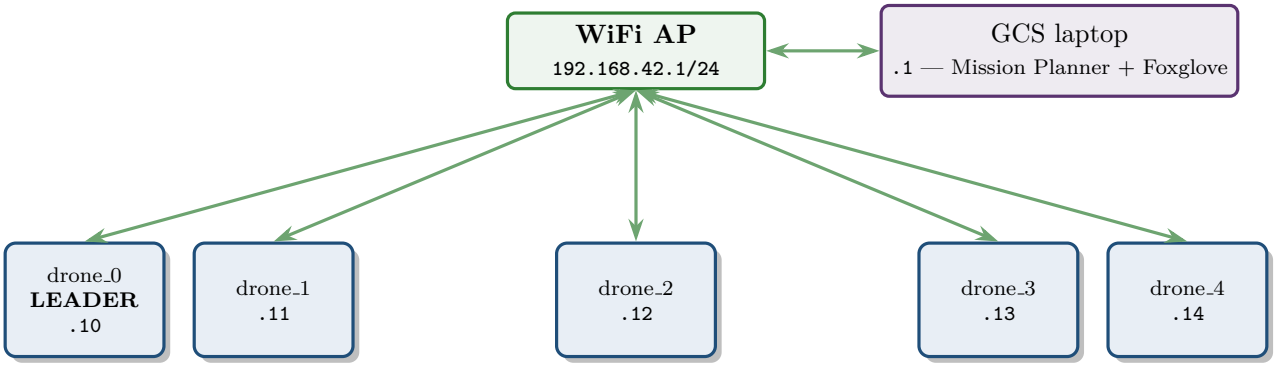
4. Stage C — The LAN and Cyclone DDS

Stage C — Two Jetsons on WiFi, talking
Goal: prove DDS discovery and per-drone Cyclone DDS config work before scaling to five.

This is the stage that SITL cannot teach. Three facts shape it:

- `network_mode: host` is mandatory — the Docker bridge cannot bridge DDS onto the physical WiFi.
- Most WiFi APs silently drop multicast. SPDP is multicast by default, so it does not arrive.
- The repo's answer is an *explicit peer list* in `config/networks/cyclonedds_drone_template.xml`.

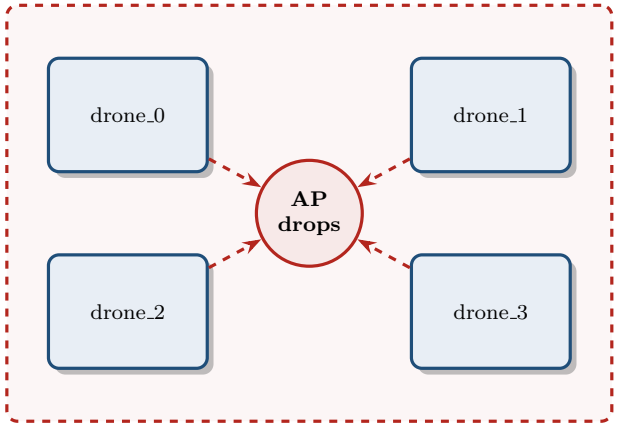
4.1. Network topology — the LAN you build



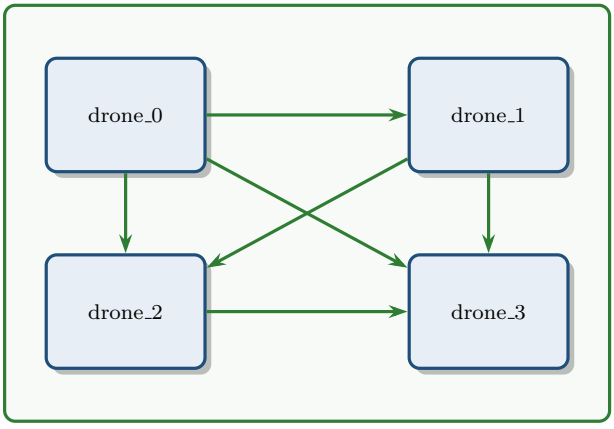
All five drones + GCS share one LAN. `ROS_DOMAIN_ID=42` on every node. Cyclone DDS peer list lists every IP — no multicast required.

4.2. Why the peer list — multicast SPDP vs unicast SPDP

Multicast SPDP — typical WiFi AP behaviour



Unicast SPDP — per-peer addressed packets



4.3. The peer-list config (per Jetson)

config/networks/cyclonedds_drone_0.xml

```
<CycloneDDS>
<Domain id="any">
<General>
<NetworkInterfaceAddress>wlan0</NetworkInterfaceAddress>
<AllowMulticast>spdp</AllowMulticast> <!-- fall back to "false" if AP hostile -->
</General>
<Discovery>
<Peers>
<Peer address="192.168.42.10"/> <!-- drone_0 (leader) -->
<Peer address="192.168.42.11"/> <!-- drone_1 -->
<Peer address="192.168.42.12"/> <!-- drone_2 -->
<Peer address="192.168.42.13"/> <!-- drone_3 -->
<Peer address="192.168.42.14"/> <!-- drone_4 -->
<Peer address="192.168.42.1"/> <!-- gcs laptop -->
</Peers>
</Discovery>
</Domain>
</CycloneDDS>
```

4.4. Two-node DDS probe before scaling

Bring up two Jetsons (any pair), *outside* the demo stack to isolate DDS from MAVROS. On each:

```
# Jetson #0:
CYCLONEDDS_URI=file://$PWD/config/networks/cyclonedds_drone_0.xml \
ROS_DOMAIN_ID=42 RMW_IMPLEMENTATION=rmw_cyclonedds_cpp \
ros2 topic pub -r1 /__dds_probe std_msgs/msg/String 'data:  hello'

# Jetson #1:
CYCLONEDDS_URI=file://$PWD/config/networks/cyclonedds_drone_1.xml \
ROS_DOMAIN_ID=42 RMW_IMPLEMENTATION=rmw_cyclonedds_cpp \
ros2 topic echo /__dds_probe
```

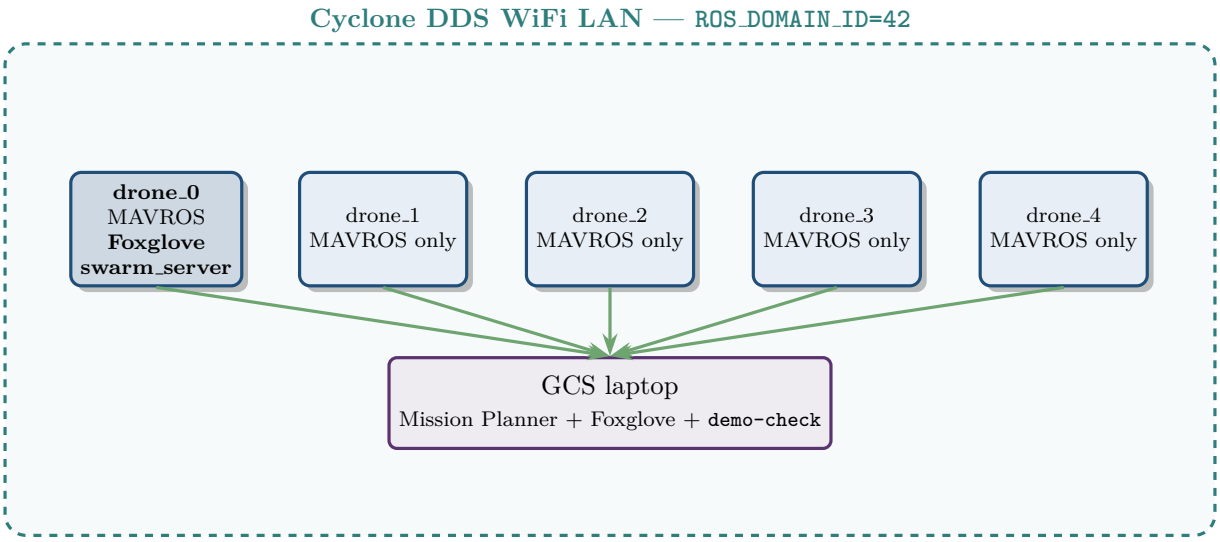
Gate to leave Stage C: `hello` flows from #0 to #1 (and back the other way). If this does not work, no demo bringup will. Most common cause: client isolation on the AP, or `NetworkInterfaceAddress` naming the wrong iface.

5. Stage D — All five Jetsons up, props OFF

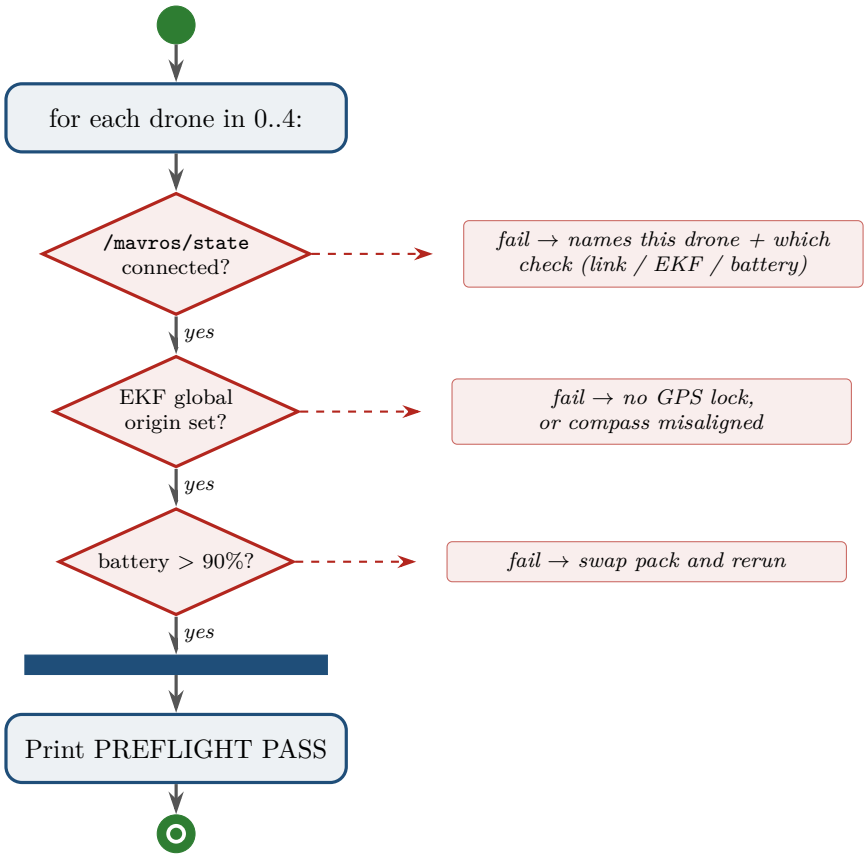
Stage D — Five companions discovered, preflight gate green

Goal: the swarm orchestrator on the leader sees and queries all four followers.

5.1. Deployment view — all five



5.2. The preflight gate — what `make demo-check` runs



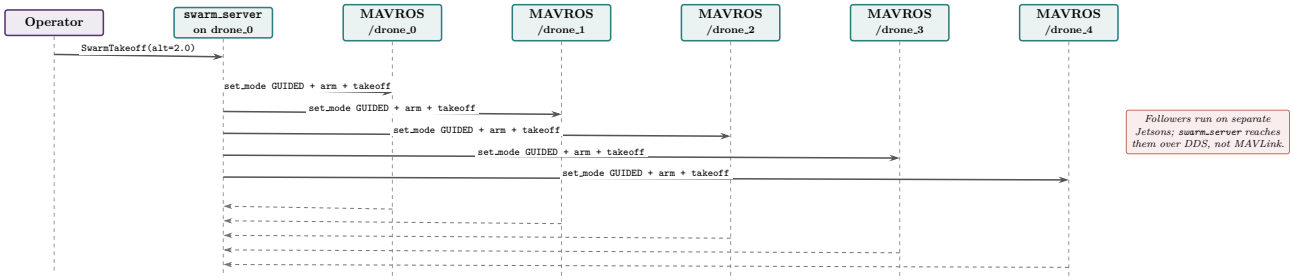
Gate to leave Stage D: `make demo-check` prints `PREFLIGHT PASS`, all five `/drone_<N>/mavros/state` reported `connected: true`, and Foxglove (`ws://192.168.42.10:8765`) shows five poses.

6. Stage E — Exercise the swarm without flying

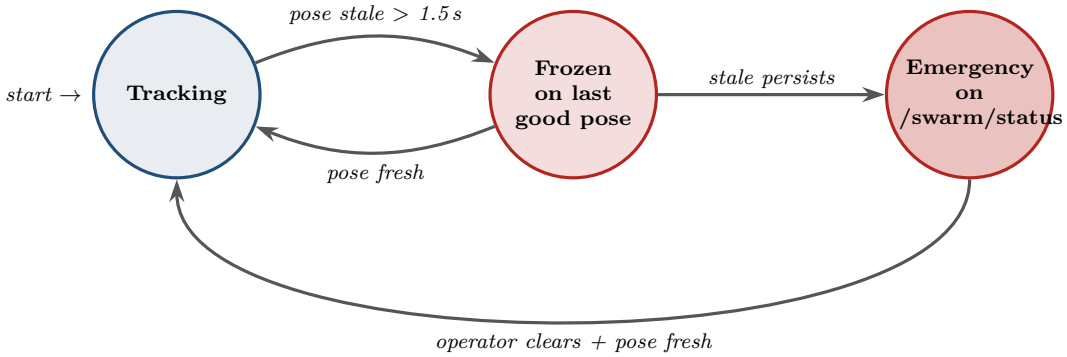
Stage E — Props OFF on all five, `/swarm/takeoff` fans out

Goal: the orchestrator reaches every follower over DDS, and the watchdog is rehearsed.

6.1. `/swarm/takeoff` fan-out — one call, five FCs



6.2. The leader-pose watchdog — state machine



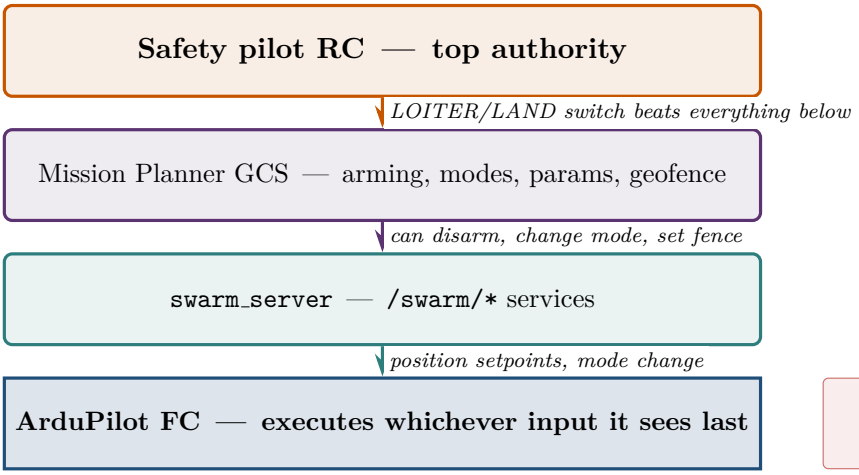
Rehearse without unplugging: `ros2 param set /swarm_server simulate_leader_dropout true`

Gate to leave Stage E: `/swarm/takeoff` switches *every* FC to GUIDED (or refuses to arm if no GPS — same on every drone). The simulated watchdog dropout raises the emergency on `/swarm/status` within 1.5 s.

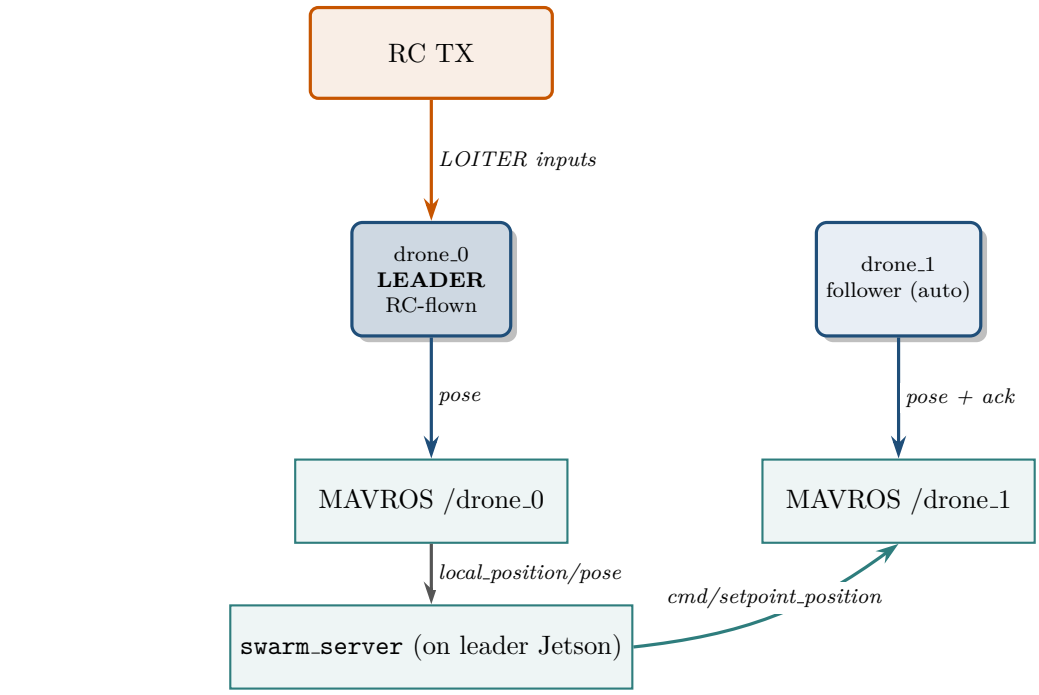
7. Stage F — First hover, then two-drone leader-follow

Stage F — Props on, one or two drones
Goal: validate the airframe and the role separation before scaling to five.

7.1. Authority hierarchy — who can override whom



7.2. Leader-follow data flow (two drones)



7.3. Two-drone leader-follow procedure

1. Props on drone_0 and drone_1 only. Drones 2–4 stay grounded with props off but powered (keep proving DDS health).

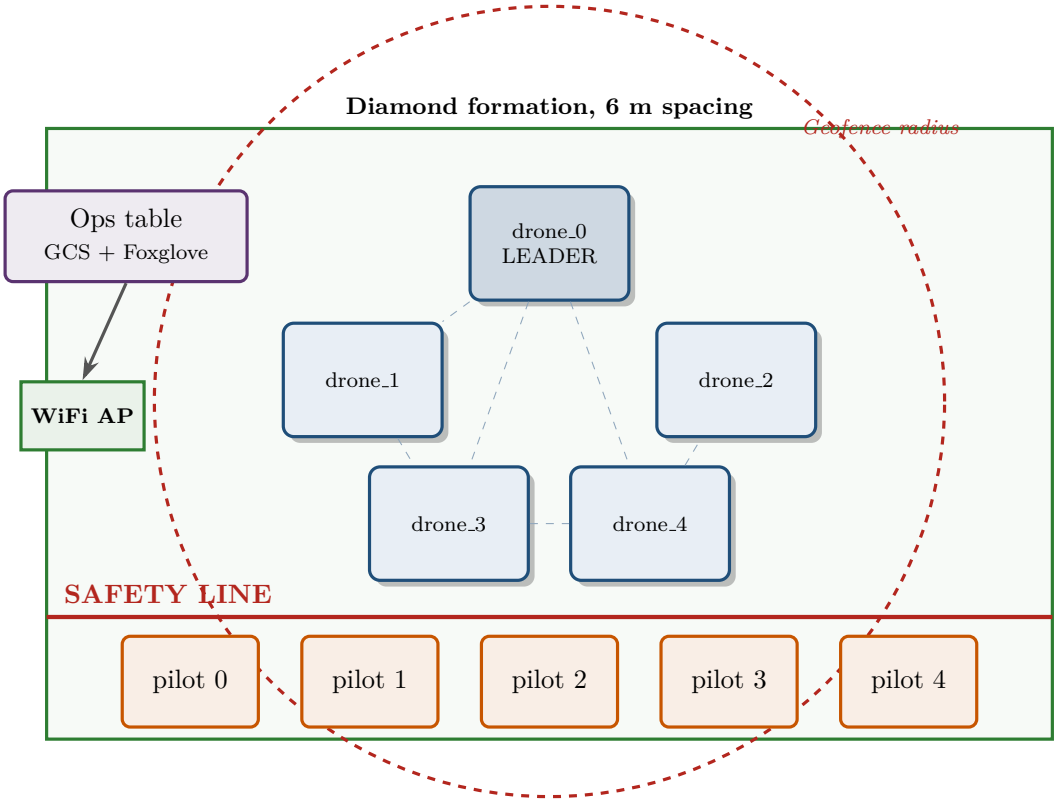
2. `make demo-check` — gate must say PREFLIGHT PASS.
3. `/swarm/takeoff alt=4.0` — both drones lift to 4 m hover in GUIDED.
4. Drone_0's safety pilot flips RC to **LOITER**. Drone_0 is now hand-flown.
5. `/swarm/follow_leader enable=true formation=line spacing=5.0` — drone_1 tracks drone_0's live pose.
6. Leader pilot flies a slow box; watch `/swarm/formation_error` on Foxglove.
7. `/swarm/follow_leader enable=false`; leader lands on RC; `/swarm/land` for drone_1.

Gate to leave Stage F: two drones complete the leader-follow box with `formation_error` remaining below your acceptable threshold (start at 2 m).

8. Stage G — Full five-drone diamond

Stage G — Five drones in the air, in formation
Goal: the end-state of Phase 2.5b.

8.1. Field setup — top-down



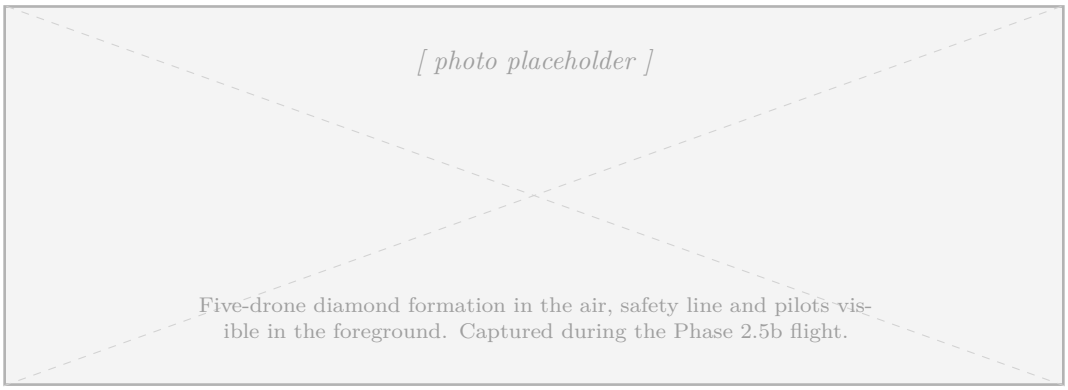
8.2. Role assignment

Role	Responsibility
Leader pilot	RC-fly drone_0 in LOITER through a slow, pre-briefed pattern. Calls abort if any follower drifts unsafely.
Follower pilots (×4)	One per drone. Hand on the transmitter. RC override to LOITER or LAND on cue or on any unsafe behaviour.
Swarm operator	Issues `/swarm/*` service calls from the ops laptop. Watches `/swarm/status` and `/swarm/formation_error`.
Safety officer	Outside the loop; calls "abort" verbally when the pre-briefed envelope is breached.

8.3. The flight, condensed

1. Preflight gate on the leader Jetson: `make demo-check` → PREFLIGHT PASS.
2. Five pilots in position, transmitters live, callouts ready.
3. `/swarm/takeoff alt=5.0`.

4. Leader pilot flips to LOITER.
5. `/swarm/follow_leader enable=true formation=diamond spacing=6.0`.
6. Leader pilot flies the pre-briefed pattern; ops watches formation error.
7. `/swarm/follow_leader enable=false`.
8. Leader lands on RC; `/swarm/land` for followers.



Five-drone diamond formation in the air, safety line and pilots visible in the foreground. Captured during the Phase 2.5b flight.

Gate to leave Stage G: all five drones complete the briefed pattern, land cleanly, no RTL triggered, no RC override used in anger. Phase 2.5b complete.

9. Failure modes mapped to gates

Each failure has exactly one stage that should catch it. If a problem first appears *after* its catching stage, you skipped the gate.

First caught at	Failure mode	Where it manifests
Stage A	TX/RX swapped, baud mismatch, getty holding port, FC unpowered	`fc_link_test.py` prints no heartbeat; `make demo-up` healthcheck times out
Stage A	Wrong L4T release (R32 instead of R36)	Base-image build fails or container exits immediately
Stage A	ArduPilot params not written / not rebooted	FC connects but rejects `set_mode GUIDED`; geofence default-off
Stage B	Overlay build broken (`swarm_msgs` IDL drift, missing dep)	Container exits before MAVROS launch; check `/tmp/overlay_log`
Stage C	AP drops multicast / has client isolation	`__dds_probe` silent from peer; topic list shows only local node
Stage C	Wrong `NetworkInterfaceAddress`	DDS picks loopback or eth, never WiFi — no peers seen
Stage C	Duplicate IP on the LAN	DDS sees half the swarm; `ros2 node list` is unstable
Stage D	Duplicate `SYSID_THISMAV`	One MAVROS silently drops; `demo-check` names the missing drone
Stage D	Compass / EKF not calibrated	EKF origin never sets; preflight blocks on that drone
Stage D	`SRn_*` stream rates wrong for the wired port	`local_position/pose` silent on that drone; follower cannot track
Stage E	Watchdog timing not what you expected	Simulated dropout does not raise emergency in 1.5 s; tune `LEADER_POSE_TIMEOUT`
Stage F	PID tune wrong for airframe	Drone oscillates in hover; abort, retune in Mission Planner
Stage F	Geofence radius too tight for formation spacing	Followers RTL on engage; widen `FENCE_RADIUS`
Stage G	Operator coordination breakdown	Safety officer abort; debrief, rehearse the choreography

10. What to do, tomorrow

Sit at the bench. One Jetson, one FC. Run Stage A end to end.
Do not move until `connected: true`.

Same Jetson, Stage B. Props off, twitch test.
`/swarm/takeoff` switches the FC to GUIDED.

Image a second Jetson. Run only the DDS probe (Stage C).
No demo stack, no MAVROS — prove DDS first.

Bring up both Jetsons under the demo stack.
Verify `ros2 topic list` shows both drones' MAVROS namespaces.

Tethered hover on drone_0, then two-drone leader-follow (Stage F).
Only now do Jetsons #3, #4, #5 get touched.

The point. The catalogue of failure modes in §8 is long. The bring-up sequence is what makes that list *shrink*: each stage takes one item off the list before the next stage's failures can hide behind it. Five drones in the air on the first try is a fluke; five drones in the air on the first try *after* stages A–F is the engineering.

References.

- `docs/architecture/end_to_end/end_to_end_setup.pdf` — architecture, UML, network topology, field setup.
- `docs/runbooks/jetson_swarm_operations.md` — per-step command reference and troubleshooting matrix.
- `docs/runbooks/first_flight.md` — the Phase 2.5b flight procedure (safety gate, roles, aborts).
- `config/networks/cyclonedds_drone_template.xml` — the DDS peer-list template (§5).
- `config/ardupilot_params/README.md` — the param-load procedure used in Stage A.
- `scripts/hardware/fc_link_test.py` — pure pymavlink bench probe (Stage A).